

Estructuras de Datos y Algoritmos

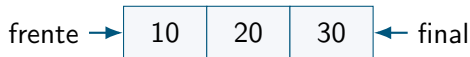
Tema 8: Colas

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

De pilas a colas

En una **pila**, el último elemento agregado sale primero: **LIFO**. En una **cola**, el primero agregado sale primero: **FIFO** (*first in, first out*).



sale 10

entra 40

Al igual que la pila, la cola es un **TDA**: fija qué operaciones ofrecemos, no cómo guardamos los elementos.

Definición: TDA cola

Una cola es una secuencia finita de elementos en la que se insertan elementos por el extremo llamado **final** y se remueven o consultan elementos por el extremo contrario llamado **frente**. Sigue el principio FIFO (*first in, first out*), lo que significa que el primer elemento agregado es el primero que se retira.

Estado de frente a final:

$$Q = (10, 20, 30).$$

El frente es 10 y el final es 30.

No hay capacidad fija en esta definición: una representación particular podría imponerla, pero no forma parte del TDA que usaremos.

Cola vacía: $Q = ()$.

No tiene frente ni final.

Operación	Efecto	Devuelve
<code>encolar(x)</code>	Agrega x al final.	None
<code>desencolar()</code>	Retira el elemento del frente.	Dato retirado
<code>frente()</code>	Consulta el frente sin retirarlo.	Dato consultado
<code>esta_vacia()</code>	Consulta si no hay elementos.	Booleano
<code>longitud()</code>	Consulta cuántos elementos hay.	Entero no negativo

No ofrecemos acceso por índice, búsqueda ni eliminación de un elemento interior. Se admiten datos repetidos y valores como None.

Sea $Q = (q_0, \dots, q_{m-1})$, escrito de **frente a final**. Si $m > 0$, el frente es q_0 .

Operación	Estado posterior Q'	Resultado
encolar(x)	$(q_0, \dots, q_{m-1}, x)$	None
desencolar()	$(q_1, \dots, q_{m-1})$	q_0
frente()	Q	q_0

Si $m = 0$, encolar x deja la cola (x) . Si $m = 1$, desencolar deja la cola vacía. En una cola vacía, frente() y desencolar() producen `IndexError` y no modifican el estado.

Cola con list

Podemos representar la cola con el frente en el índice 0. Los métodos `esta_vacia()` y `longitud()` pueden reciclarse de Pilas.

```
class ColaLista:
    def __init__(self, lista_datos = []):
        self._datos = lista_datos
    def encolar(self, x):
        self._datos.append(x)
    def desencolar(self):
        if self.esta_vacia():
            raise IndexError("cola vacia")
        return self._datos.pop(0)
    def frente():
        if self.esta_vacia():
            raise IndexError("cola vacia")
        return self._datos[0]
```

Estado: [10, 20, 30]

10	20	30
----	----	----

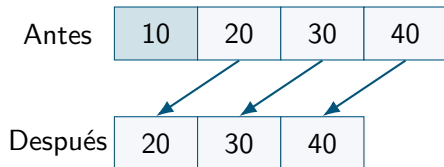
frente

final

¿Qué costo tiene `pop(0)`?

Desplazamiento

Al retirar el primer elemento de una `list`, las referencias restantes deben avanzar una posición.



Con n elementos, `pop(0)` desplaza $n - 1$ referencias: $\Theta(n)$. Encolar con `append()` tiene costo amortizado $\Theta(1)$.

deque en Python

`collections.deque` permite agregar por la derecha y retirar por la izquierda sin desplazar todos los elementos.

```
from collections import deque
class ColaDeque:
    def __init__(self):
        self._datos = deque()
    def encolar(self, x):
        self._datos.append(x)
    def desencolar(self):
        return self._datos.popleft()
```

`popleft()` produce `IndexError` si la cola está vacía.

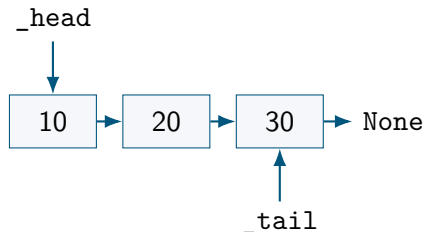
`frente()` consultaría `_datos[0]`.
`esta_vacia()` y `longitud()` se obtienen con `len()`.

Cola enlazada

Reutilizamos `Nodo(dato, siguiente)` del tema de listas. Guardamos referencias a los dos extremos.

```
class ColaEnlazada:
    def __init__(self):
        self._head = None
        self._tail = None
        self._n = 0
    def esta_vacia(self):
        return self._head is None
    def longitud(self):
        return self._n
```

Estado abstracto: (10, 20, 30).



La cabeza señala el frente; la cola, el final.

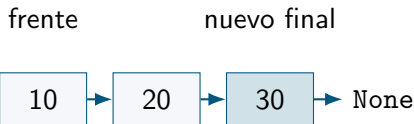
Encolar con nodos

Método de ColaEnlazada. El nuevo nodo siempre será el final.

```
def encolar(self, x):  
    nuevo = Nodo(x, None)  
    if self._tail is None:  
        self._head = nuevo  
    else:  
        self._tail.siguiente = nuevo  
    self._tail = nuevo  
    self._n += 1
```

Si estaba vacía: `_head` y `_tail` señalan el mismo nodo.

Si tenía elementos: el final anterior enlaza con nuevo; después avanzamos `_tail`.



Desencolar con nodos

Métodos de ColaEnlazada. Retiramos el nodo del frente.

```
def desencolar(self):
    if self._head is None:
        raise IndexError("cola vacia")
    dato = self._head.dato
    self._head = self._head.siguiente
    if self._head is None:
        self._tail = None
    self._n -= 1
    return dato

def frente(self):
    if self._head is None:
        raise IndexError("cola vacia")
    return self._head.dato
```

Varios nodos: el segundo pasa a ser el frente.

Un nodo: al avanzar, `_head` queda en `None`. También debemos poner `_tail` en `None`.

`frente()` consulta sin modificar enlaces ni contador.

Sea n el número de elementos. En la tabla, $()$ indica costo amortizado.

Operación	list	deque	Enlazada
Encolar	$\Theta(1)$ $()$	$O(1)$ $*$	$\Theta(1)$
Desencolar	$\Theta(n)$	$O(1)$ $*$	$\Theta(1)$
Consultar frente	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Consultar longitud	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Consultar si está vacía	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

deque es la elección habitual en Python para una cola FIFO.

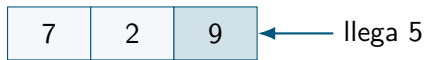
Ejemplo: media móvil

Dada una sucesión

$$x_1, x_2, \dots, x_n$$

queremos calcular, para cada posición, el promedio de los últimos k valores:

$$M_i = \frac{x_{i-k+1} + \dots + x_i}{k}, \quad i \geq k.$$



más antiguo más reciente

La cola guarda exactamente la ventana actual de tamaño a lo más k .

Algoritmo con cola

La idea es no recalcular la suma completa de la ventana en cada paso.

MediaMovil

Inicializar una cola vacía Q y una suma $s \leftarrow 0$.

Para $i = 1, 2, \dots, n$:

 Encolar x_i en Q

$s \leftarrow s + x_i$

 Si longitud(Q) $> k$:

$v \leftarrow \text{desencolar}(Q)$

$s \leftarrow s - v$

 Si longitud(Q) $= k$:

 Mostrar s/k

Uso de FIFO: el valor que debe salir es siempre el más antiguo de la ventana.

Traza del algoritmo

Para la sucesión (4, 7, 2, 9, 5, 8) y $k = 3$:

Dato nuevo	Sale	Cola posterior	Media mostrada
4	–	(4)	–
7	–	(4, 7)	–
2	–	(4, 7, 2)	13/3
9	4	(7, 2, 9)	18/3
5	7	(2, 9, 5)	16/3
8	2	(9, 5, 8)	22/3

Cada nuevo dato entra por el final; cuando sobra un dato, sale el frente.